

Шпаргалка по регулярным выражениям

Метасимволы, классы, повторители, группы и проверки вокруг совпадения, флаги, а также особенности Python, greg и командной строки. Синтаксис — PCRE-совместимый, он же используется в Python, JavaScript, Go, PHP и grep -E с небольшими отличиями.

Одиночные символы

- .
Любой символ, кроме перевода строки (с флагом s — любой)
- \d / \D
Цифра / не цифра
- \w / \W
Буква, цифра или подчёркивание / всё остальное
- \s / \S
Пробельный символ (пробел, табуляция, перевод строки) / непобельный
- \t \n \r
Табуляция, перевод строки, возврат каретки
- \.
Экранирование: точка как обычный символ.
Экранировать нужно . ^ \$ * + ? () [] { } | \ /

Классы символов

- [abc]
Любой из перечисленных символов
- [^abc]
Любой символ, кроме перечисленных
- [a-z0-9]
Диапазоны: строчные латинские буквы и цифры
- [а-яёА-ЯЁ]
Кириллица; буква ё в диапазон а-я не входит, её добавляют отдельно
- [.]
Внутри класса большинство метасимволов теряют особый смысл

Повторители

- *
Ноль или больше повторений
- +
Одно или больше
- ?
Ноль или одно — необязательная часть
- {3}
Ровно три повторения
- {2,5} / {2,}
От двух до пяти / два и больше
- *? +? {2,5}?
Ленивый вариант: берёт минимально возможное
По умолчанию повторители жадные: <.+> на строке <a> захватит всё целиком, а <.+?> — только <a>.

Границы

- ^ / \$
Начало и конец строки (с флагом m — каждой строки текста)
- \b / \B
Граница слова / не граница: \bkот\b не совпадёт внутри слова «котлета»
- \A / \Z
Начало и конец всего текста независимо от флага m

Группы и альтернативы

- (abc)
Группа с захватом: попадает в результат под номером 1, 2, ...
- (?:abc)
Группировка без захвата — когда нужны только скобки
- (?P<year>\d{4})
Именованная группа (в JavaScript и Go: (?<year>...))
- a|b
Или: (кот|пёс)
- \1
Ссылка на текст, уже пойманный первой группой

Проверки вокруг совпадения

- (?=...)
Дальше идёт: \d+(?= №) найдёт число перед знаком рубля, сам знак в совпадение не войдёт
- (?!...)
Дальше не идёт: foo(?!bar)
- (?<=...)
Перед этим было: (?<=id=)\d+
- (?<!...)
Перед этим не было
- Проверки ничего не потребляют: позиция в тексте не сдвигается, а совпадением считается только основная часть.

Флаги

- i
Без учёта регистра
- m
^ и \$ работают на каждой строке многострочного текста
- s
Точка совпадает и с переводом строки
- x
Пробелы и комментарии внутри выражения игнорируются — удобно для длинных шаблонов
- (?i)шаблон
Флаг прямо в выражении, если задать его отдельно нельзя

Python

```
import re

m = re.search(r"(?P<user>\w+)@(\w+\.\w+)", text)
if m:
    m.group(0)      # всё совпадение
    m.group("user")  # именованная группа
    m.group(2)      # вторая группа

re.findall(r"\d+", text)      # список совпадений
re.sub(r"\s+", " ", text)     # замена
re.split(r"[;,\s*]", text)    # разбиение
pat = re.compile(r"^\d{4}-\d{2}$") # заранее скомпилировать
```

`r"..."`
Сырая строка: обратные слешы не нужно удваивать

`re.match` / `re.search` / `re.fullmatch`
Только с начала / где угодно / вся строка целиком

`re.IGNORECASE` | `re.MULTILINE`
Флаги передаются отдельным аргументом

Командная строка

`grep -E "^error.*timeout" app.log`
Расширенный синтаксис: без `-E` нужно экранировать `+ ? { } ( ) |`

`grep -P "\d{3}-\d{2}"`
Синтаксис PCRE, если `grep` собран с его поддержкой

`grep -o / -i / -v / -c`
Вывести только совпадение / без учёта регистра / инвертировать / посчитать

`rg "TODO|FIXME" -t py`
`ripgrep`: PCRE-подобный синтаксис по умолчанию и фильтр по типу файлов

`sed -E "s/[0-9]+/N/g" file`
Замена; & в правой части — весь найденный текст, \1 — группа

Готовые примеры

`^\d{4}-\d{2}-\d{2}$`
Дата в формате ГГГГ-ММ-ДД

`^\+?[0-9][0-9\s\-\(\)]{7,}$`
Телефон в свободной записи

`\b(?:\d{1,3}\.){3}\d{1,3}\b`
IPv4-адрес (без проверки диапазона 0–255)

`https?:\/\/[^\s"'>]+`
Ссылка в тексте

`^[^\s]+@[^\s]+\.[^\s]{2,}$`
Простая проверка адреса почты: полный разбор по стандарту практически невозможен

`(?m)^\s*$\n`
Пустые строки — для удаления заменой

Что помнить

- Проверяйте шаблон на реальных данных, включая неудачные случаи — сервисы вроде `regex101` показывают разбор по шагам.
- Вложенные повторители `((a)+)` на длинной строке способны надолго подвесить движок; упрощайте шаблон.
- Разметку HTML и XML регулярными выражениями разбирать не стоит — берите парсер.
- Длинный шаблон с флагом `x` и комментариями читается заметно лучше однострочного.
- Кириллица требует поддержки Unicode: в Python это поведение по умолчанию, в других инструментах может потребоваться флаг.